GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 4

Estrategias de mantenimiento y evolución

Enunciado de las actividades 4 y 5 · Maestría en Ingeniería de Software, Universidad de La Sabana

El encargo

> «Ya sabemos qué le pasa al sistema, ya controlamos sus cambios y ya entregamos con un pipeline. Ahora queremos que sea mejor y queremos verlo en números, no en opiniones.»
>
> Responsable del sistema que eligió cada grupo.

Es la unidad que cierra el proyecto del curso. El sistema es el mismo de las unidades anteriores. Las mejoras se eligen a partir del diagnóstico de la Unidad 1 y se implementan con el pipeline de la Unidad 3 como red de seguridad. La presentación final cuenta el proyecto completo.

Actividad 4 · Propuesta e implementación de estrategias de mantenimiento y evolución

Modalidad	Grupal, de 2 a 3 estudiantes · calificable, **25 %**
Dedicación	20 horas
Entregables	**1.** Un documento técnico de **8 a 10 páginas** o un **README.md**. **2.** El código o las evidencias de la implementación. **3.** Los resultados de la mejora: el antes y el después.
Reto	¿Cómo mejorar la calidad y sostenibilidad de un sistema de software mediante estrategias de mantenimiento y evolución?
Instrumento	Rúbrica, publicada en el aula virtual

Qué debe tener el documento

#	Sección	Qué debe mostrar	Formato
1	Oportunidades de mejora	Las mejoras elegidas, de dónde salen (deuda, riesgos, métricas de las unidades anteriores) y por qué esas y no otras	F-22
2	Estrategia	Refactorizar, modernizar por partes o reemplazar, con la justificación de cada decisión	F-23
3	Implementación	Las pruebas de caracterización, los pasos y sus commits, y lo que el pipeline verificó en cada uno	F-24
4	Medición	El objetivo de cada mejora, las métricas elegidas y su valor antes de empezar	F-25
5	Evaluación del impacto	El antes y el después, qué mejoró, qué no y por qué	F-26

Criterios de evaluación

La rúbrica tiene cinco criterios. Los niveles y los puntajes están en el aula virtual.

Criterio	Qué se observa en el nivel más alto
Identificación de mejoras	Identificación clara y bien justificada
Propuesta de solución	Solución robusta, viable e innovadora
Implementación de mejoras	Implementación completa y funcional
Evaluación del impacto	Evidencia clara de mejora del sistema
Documentación y soporte	Documento claro y bien argumentado

Actividad 5 · Presentación final del proyecto

Modalidad	Grupal · calificable, **10 %**
Dedicación	10 horas
Entregables	**1.** Una presentación de **10 a 15 diapositivas**. **2.** Una demo funcional del sistema. **3.** Un resumen ejecutivo de **2 a 3 páginas** o un README.md.
Reto	¿Cómo demostrar la efectividad de una solución integral de gestión de configuración y mantenimiento del software?
Estructura	Problema, solución, implementación y resultados (formato F-27)

Criterios de la rúbrica: estructura de la presentación, dominio del tema, demostración, argumentación técnica, y presentación y comunicación. Los niveles y los puntajes están en el aula virtual.

Paso a paso

1. **Elegir dos o tres mejoras** (F-22) a partir del F-05, el F-06 y el F-20.
2. **Decidir la estrategia** de cada una (F-23). El interactivo *¿Reemplazar o evolucionar?* ayuda a ordenar la discusión.
3. **Declarar el objetivo y medir el antes** (F-25) en una versión identificada con una etiqueta.
4. **Escribir las pruebas de caracterización** del código que se va a tocar.
5. **Implementar en pasos pequeños** (F-24), un commit por paso, por PR, con la CI en verde.
6. **Medir el después y evaluar** (F-26).
7. **Preparar la presentación final** y su demo (F-27), **verificar** con el F-28 y entregar.

```
git tag -a antes-de-mejoras -m "Línea base de medición"
python herramientas/impacto.py <sistema> --antes antes-de-mejoras --despues HEAD --ruta src --
pruebas tests
```

Material de la unidad

- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.ª ed.). Addison-Wesley. Capítulos 1 y 2. Disponible en la biblioteca.
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering*. O'Reilly Media. Capítulos 1 a 3. Disponible en la biblioteca.
- ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 14764:2022 Software engineering — Software life cycle processes — Maintenance*. Secciones principales.
- Complementarios: Prometheus, *Documentation*, https://prometheus.io/docs/; Grafana Labs, *Grafana documentation*, https://grafana.com/docs/

Errores frecuentes

- **Medir después de cambiar.** El antes solo se puede medir antes.
- **Refactorizar y corregir en el mismo commit.** Después nadie sabe qué cambió el comportamiento.
- **Diez mejoras a medias.** Dos terminadas, medidas y explicadas valen más.
- **Una sola métrica.** Puede mejorar mientras el sistema empeora en otra cosa. Declaren el objetivo y miren varias.
- **Una demo que depende de que todo salga bien en vivo.** Ensáyenla y tengan grabada la ejecución del pipeline.